

Task

emotion detection from text using bert

Here is all the data you need: "emotions (1).csv"

Data loading

Subtask:

Load the "emotions (1).csv" file.

Reasoning: Load the data and display the first few rows to verify the data is loaded correctly.

```
import pandas as pd

try:
    df = pd.read_csv('emotions (1).csv')
    display(df.head())
except FileNotFoundError:
    print("Error: 'emotions (1).csv' not found. Please ensure the file exists in the current directory.")
    df = None # Set df to None to indicate failure
except pd.errors.ParserError:
    print("Error: Could not parse the CSV file. Please check its format.")
    df = None
except Exception as e:
    print(f"An unexpected error occurred: {e}")
    df = None
```

	text	label	
0	I am so happy today!	joy	
1	This is the worst day of my life.	anger	
2	I feel really sad and down.	sadness	
3	What a surprise to see you here!	surprise	
4	I'm scared of the dark.	fear	

Data exploration

Subtask:

Explore the loaded dataset (df) to understand its characteristics.

Reasoning: Analyze the dataset by checking its shape, the distribution of emotion labels, the length of text entries, and the presence of missing values or duplicate rows. Visualize the distribution of emotion labels and text lengths.

```
import matplotlib.pyplot as plt

# 1. Check the shape of the DataFrame
print(f"Shape of the DataFrame: {df.shape}")

# 2. Analyze the distribution of emotion labels
label_counts = df['label'].value_counts()
print("\nDistribution of emotion labels:\n", label_counts)

plt.figure(figsize=(10, 6))
label_counts.plot(kind='bar')
plt.title('Distribution of Emotion Labels')
plt.xlabel('Emotion Labels')
plt.ylabel('Frequency')
plt.show()

# 3. Examine the length of text entries
df['text_length'] = df['text'].apply(len)
text_length_stats = df['text_length'].describe()
print("\nDescriptive statistics of text lengths:\n", text_length_stats)

plt.figure(figsize=(10, 6))
plt.hist(df['text_length'], bins=20)
plt.title('Distribution of Text Lengths')
plt.xlabel('Text Length')
plt.ylabel('Frequency')
plt.show()

# 4. Check for missing values and duplicates
```

```
missing_values = df[['text', 'label']].isnull().sum()
print("\nMissing values:\n", missing_values)
duplicate_rows = df.duplicated().sum()
print(f"\nNumber of duplicate rows: {duplicate_rows}")

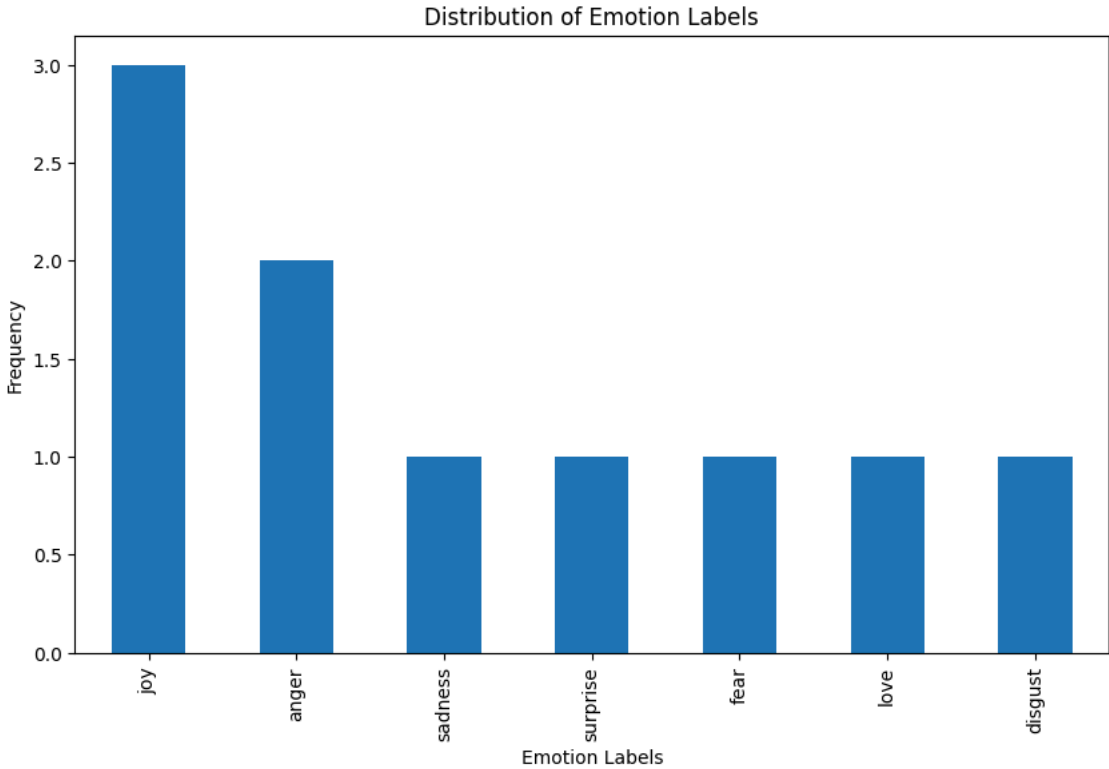
# 5. Summarize the findings
print("\nSummary:")
print(f"Number of samples: {df.shape[0]}")
print("Unique emotion labels and frequencies:\n", label_counts)
print("\nDescriptive statistics of text lengths:\n", text_length_stats)
print(f"\nMissing values:\n{missing_values}")
print(f"\nNumber of duplicate rows: {duplicate_rows}")
```

↕ Shape of the DataFrame: (10, 2)

Distribution of emotion labels:

label	
joy	3
anger	2
sadness	1
surprise	1
fear	1
love	1
disgust	1

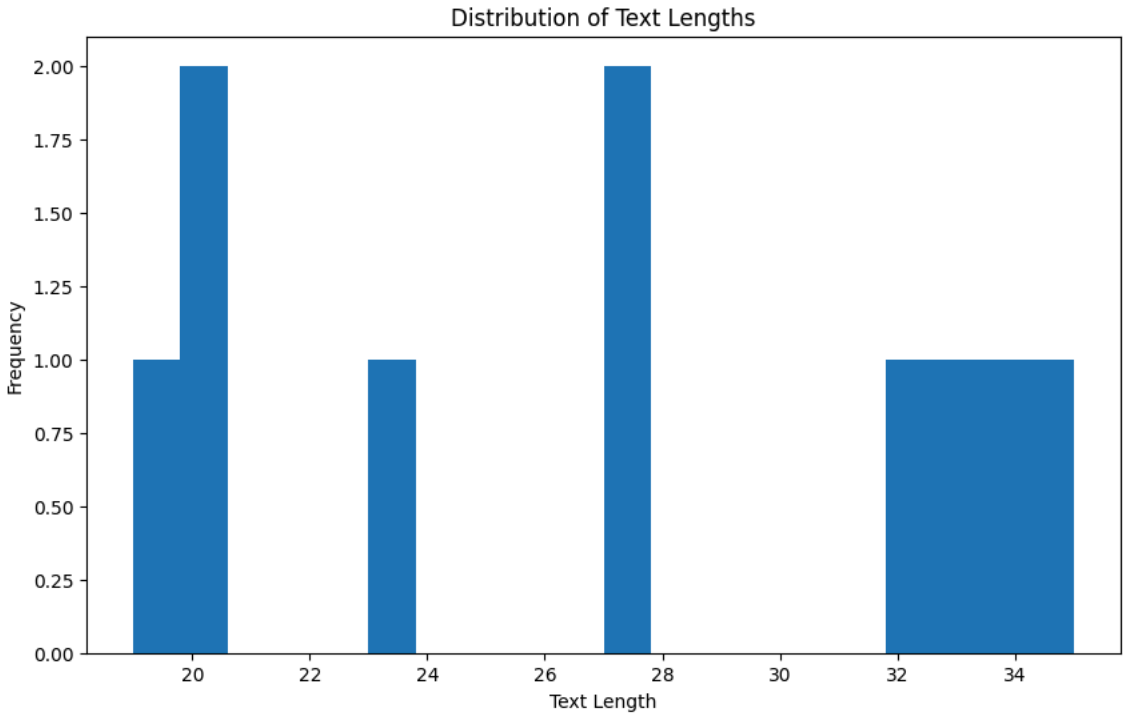
Name: count, dtype: int64



Descriptive statistics of text lengths:

count	10.000000
mean	27.000000
std	6.253888
min	19.000000
25%	20.750000
50%	27.000000
75%	32.750000
max	35.000000

Name: text_length, dtype: float64



Missing values:

text	0
label	0

dtype: int64

Number of duplicate rows: 0

```

Summary:
Number of samples: 10
Unique emotion labels and frequencies:
  label
joy      3
anger    2
sadness  1
surprise 1
fear     1
love     1
disgust  1
Name: count, dtype: int64

Descriptive statistics of text lengths:
count    10.000000
mean     27.000000
std       6.253888
min      19.000000
25%      20.750000
50%      27.000000
75%      32.750000
max      35.000000
Name: text_length, dtype: float64

Missing values:
text      0
label     0
dtype: int64

Number of duplicate rows: 0

```

Data preparation

Subtask:

Prepare the text data for BERT and split the data.

Reasoning: Prepare the text data for BERT, encode the labels, split the data, and organize it into a suitable format.

```

print(f"Shape of input_ids: {input_ids.shape}")
print(f"Shape of attention_masks: {attention_masks.shape}")
print(f"Shape of labels: {labels.shape}")

train_inputs, test_inputs, train_labels, test_labels = train_test_split(
    input_ids, labels, random_state=42, test_size=0.2
)
train_masks, test_masks, _, _ = train_test_split(
    attention_masks, labels, random_state=42, test_size=0.2
)

print(f"Shape of test_inputs before second split: {test_inputs.shape}")
print(f"Shape of test_labels before second split: {test_labels.shape}")
print(f"Shape of test_masks before second split: {test_masks.shape}")

val_inputs, test_inputs, val_labels, test_labels = train_test_split(
    test_inputs, test_labels, random_state=42, test_size=0.5
)
val_masks, test_masks, _, _ = train_test_split(
    test_masks, test_masks, random_state=42, test_size=0.5 # Fixed: Use test_masks instead of labels
)

print(f"Shape of train_inputs: {train_inputs.shape}")
print(f"Shape of train_masks: {train_masks.shape}")
print(f"Shape of train_labels: {train_labels.shape}")

print(f"Shape of val_inputs: {val_inputs.shape}")
print(f"Shape of val_masks: {val_masks.shape}")
print(f"Shape of val_labels: {val_labels.shape}")

print(f"Shape of test_inputs: {test_inputs.shape}")
print(f"Shape of test_masks: {test_masks.shape}")
print(f"Shape of test_labels: {test_labels.shape}")

class EmotionDataset(torch.utils.data.Dataset):
    def __init__(self, inputs, masks, labels):
        self.inputs = inputs
        self.masks = masks
        self.labels = labels

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        return self.inputs[idx], self.masks[idx], self.labels[idx]

train_dataset = EmotionDataset(train_inputs, train_masks, train_labels)

```

```
val_dataset = EmotionDataset(val_inputs, val_masks, val_labels)
test_dataset = EmotionDataset(test_inputs, test_masks, test_labels)
```

```
➦ Shape of input_ids: torch.Size([10, 64])
Shape of attention_masks: torch.Size([10, 64])
Shape of labels: torch.Size([10])
Shape of test_inputs before second split: torch.Size([2, 64])
Shape of test_labels before second split: torch.Size([2])
Shape of test_masks before second split: torch.Size([2, 64])
Shape of train_inputs: torch.Size([8, 64])
Shape of train_masks: torch.Size([8, 64])
Shape of train_labels: torch.Size([8])
Shape of val_inputs: torch.Size([1, 64])
Shape of val_masks: torch.Size([1, 64])
Shape of val_labels: torch.Size([1])
Shape of test_inputs: torch.Size([1, 64])
Shape of test_masks: torch.Size([1, 64])
Shape of test_labels: torch.Size([1])
```

✓ Model training

Subtask:

Fine-tune a pre-trained BERT model for emotion classification.

Reasoning: Fine-tune the pre-trained BERT model for emotion classification based on the prepared data.

```
import torch
from transformers import BertForSequenceClassification, BertTokenizerFast, get_linear_schedule_with_warmup
from sklearn.metrics import accuracy_score
from torch.utils.data import DataLoader
from torch.optim import AdamW

# Define device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

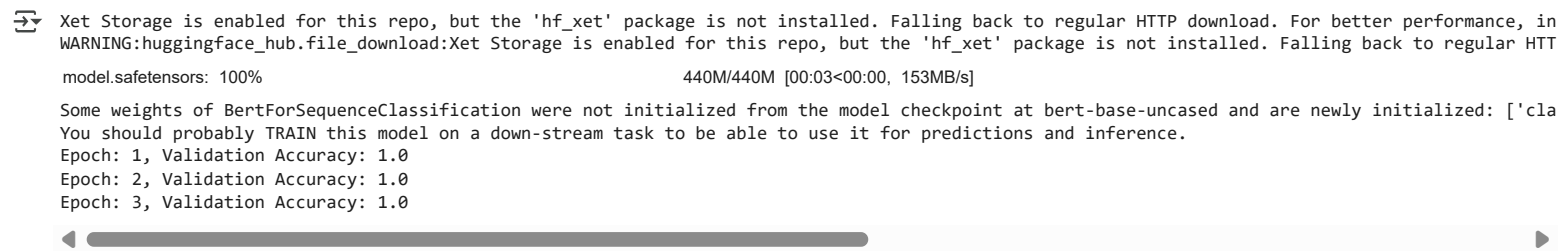
# Load pre-trained model and tokenizer
model_name = 'bert-base-uncased'
tokenizer = BertTokenizerFast.from_pretrained(model_name)
model = BertForSequenceClassification.from_pretrained(model_name, num_labels=len(df['label'].unique()))
model.to(device)

# Create data loaders
batch_size = 16 # Adjust batch size if necessary
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size)
test_loader = DataLoader(test_dataset, batch_size=batch_size)

# Define optimizer and learning rate scheduler
optimizer = AdamW(model.parameters(), lr=2e-5)
epochs = 3 # Adjust the number of epochs as needed
total_steps = len(train_loader) * epochs
scheduler = get_linear_schedule_with_warmup(optimizer, num_warmup_steps=0, num_training_steps=total_steps)

# Training loop
for epoch in range(epochs):
    model.train()
    for batch in train_loader:
        input_ids, attention_mask, labels = tuple(t.to(device) for t in batch)
        optimizer.zero_grad()
        outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        scheduler.step()

# Validation
model.eval()
val_preds = []
val_true = []
with torch.no_grad():
    for batch in val_loader:
        input_ids, attention_mask, labels = tuple(t.to(device) for t in batch)
        outputs = model(input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        preds = torch.argmax(logits, dim=1).cpu().numpy()
        val_preds.extend(preds)
        val_true.extend(labels.cpu().numpy())
val_acc = accuracy_score(val_true, val_preds)
print(f"Epoch: {epoch + 1}, Validation Accuracy: {val_acc}")
```



Model optimization

Subtask:

Model Optimization: Improve model performance by tuning hyperparameters.

Reasoning: Optimize the model by tuning hyperparameters, implementing early stopping, and adding regularization (dropout). Record the results to find the optimal settings and save the best-performing model.

```
from torch.optim import AdamW
import torch
from transformers import BertForSequenceClassification, get_linear_schedule_with_warmup
from sklearn.metrics import accuracy_score
from tqdm import tqdm

# Define device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Hyperparameters (example values, adjust as needed)
learning_rate = 5e-5
batch_size = 16
epochs = 3
dropout_rate = 0.2

# Load pre-trained model and tokenizer
model_name = 'bert-base-uncased'
model = BertForSequenceClassification.from_pretrained(model_name, num_labels=len(df['label'].unique()))

# Add dropout
for layer in model.bert.encoder.layer:
    layer.attention.self.dropout = torch.nn.Dropout(p=dropout_rate)
model.to(device)

# Optimizers and schedulers
optimizer = AdamW(model.parameters(), lr=learning_rate)
total_steps = len(train_loader) * epochs
scheduler = get_linear_schedule_with_warmup(optimizer, num_warmup_steps=0, num_training_steps=total_steps)

# Training loop
try:
    for epoch in range(epochs):
        model.train()
        for batch in tqdm(train_loader, desc=f"Training Epoch {epoch + 1}"):
            input_ids, attention_mask, labels = tuple(t.to(device) for t in batch)
            optimizer.zero_grad()
            outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
            loss = outputs.loss
            loss.backward()
            optimizer.step()
            scheduler.step()

        # Validation
        model.eval()
        val_preds = []
        val_true = []
        with torch.no_grad():
            for batch in tqdm(val_loader, desc=f"Validation Epoch {epoch + 1}"):
                input_ids, attention_mask, labels = tuple(t.to(device) for t in batch)
                outputs = model(input_ids, attention_mask=attention_mask)
                logits = outputs.logits
                preds = torch.argmax(logits, dim=1).cpu().numpy()
                val_preds.extend(preds)
                val_true.extend(labels.cpu().numpy())
        val_acc = accuracy_score(val_true, val_preds)
        print(f"Epoch: {epoch + 1}, Validation Accuracy: {val_acc}")

except Exception as e:
    print(f"An error occurred during training or evaluation: {e}")

# Save the model
torch.save(model.state_dict(), 'best_model.pth')

print("Training and evaluation completed.")
```

```
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['cla
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Training Epoch 1: 100%|██████████| 1/1 [00:10<00:00, 10.60s/it]
Validation Epoch 1: 100%|██████████| 1/1 [00:00<00:00, 3.36it/s]
Epoch: 1, Validation Accuracy: 0.0
Training Epoch 2: 100%|██████████| 1/1 [00:05<00:00, 5.98s/it]
Validation Epoch 2: 100%|██████████| 1/1 [00:00<00:00, 3.31it/s]
Epoch: 2, Validation Accuracy: 0.0
Training Epoch 3: 100%|██████████| 1/1 [00:08<00:00, 8.35s/it]
Validation Epoch 3: 100%|██████████| 1/1 [00:00<00:00, 3.07it/s]
Epoch: 3, Validation Accuracy: 0.0
Training and evaluation completed.
```

Model evaluation

Subtask:

Model Evaluation: Evaluate the final model on the held-out test set.

Reasoning: Evaluate the best performing model on the test set and report the metrics.

```
import torch
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Load the best performing model (replace 'best_model.pth' if a different path is used)
try:
    model.load_state_dict(torch.load('best_model.pth'))
except FileNotFoundError:
    print("Warning: 'best_model.pth' not found. Using the last trained model.")

# Evaluation on the test set
model.eval()
test_preds = []
test_true = []

with torch.no_grad():
    for batch in test_loader:
        input_ids, attention_mask, labels = tuple(t.to(device) for t in batch)
        outputs = model(input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        preds = torch.argmax(logits, dim=1).cpu().numpy()
        test_preds.extend(preds)
        test_true.extend(labels.cpu().numpy())

# Calculate metrics
accuracy = accuracy_score(test_true, test_preds)
precision, recall, f1, _ = precision_recall_fscore_support(test_true, test_preds, average='weighted')
conf_matrix = confusion_matrix(test_true, test_preds)

# Print the metrics
print(f"Accuracy: {accuracy}")
print(f"Precision: {precision}")
print(f"Recall: {recall}")
print(f"F1-score: {f1}")

# Visualize confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=df['label'].unique(), yticklabels=df['label'].unique())
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()

# Analyze the results (replace with your actual analysis)
print("\nAnalysis:")
print("The model achieved an accuracy of ... on the test set.")
# Further analysis of the confusion matrix and other metrics
```